

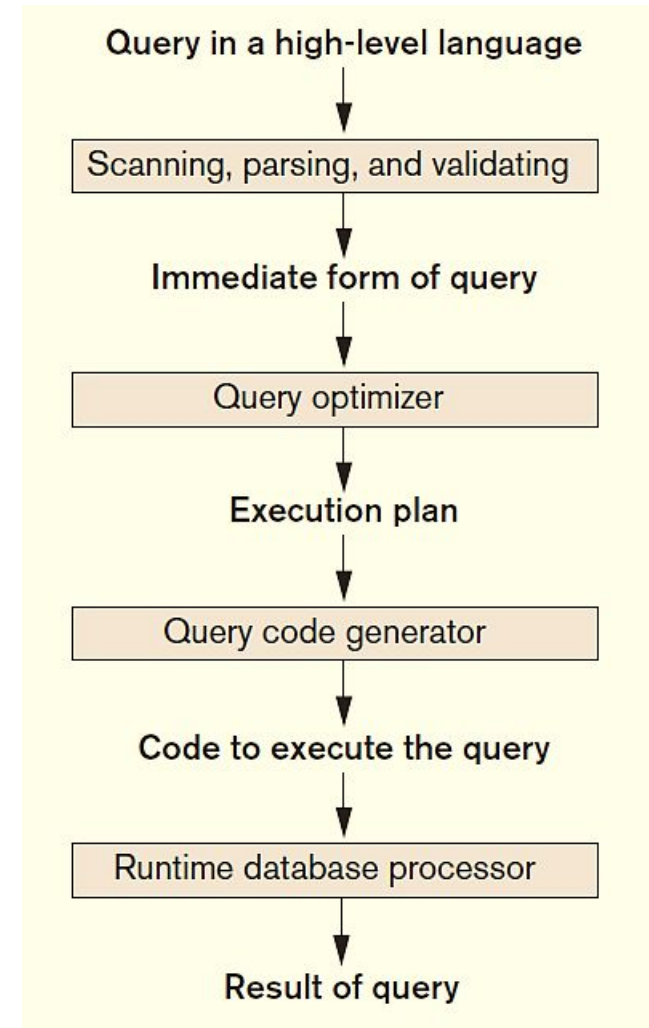
CSE 370: Database Systems

Query Processing

Introduction

A database management system (DBMS) needs to undertake a complex sequence of activities to retrieve the results corresponding to the SQL query sends from the user application. The process of retrieving the results for an input query is called query processing. The typical steps of query processing are:

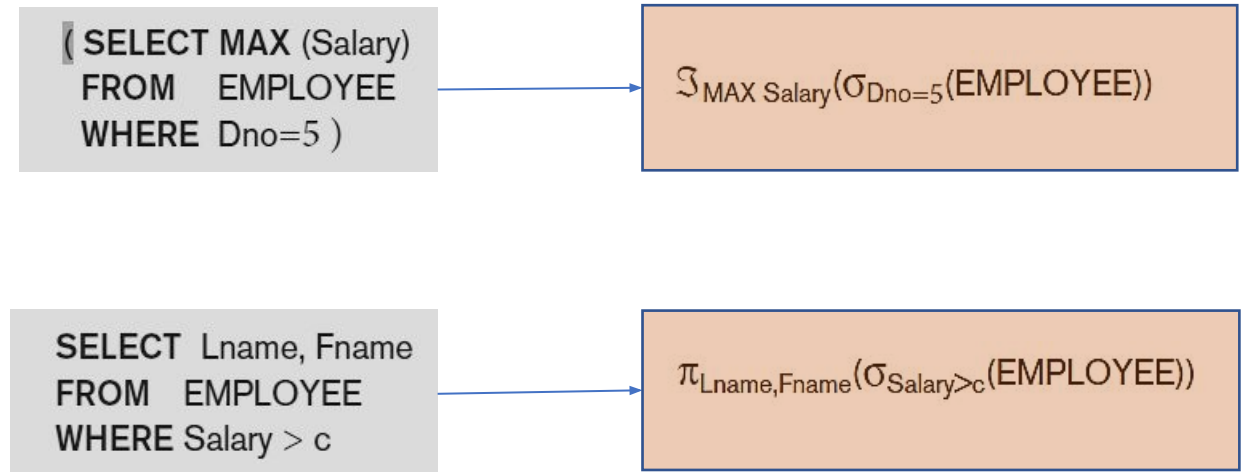
1. **Query Compilation:** involving reading, validating, and subsequently translating the SQL query into a relational algebra statement.
2. **Query Optimization:** determining the best way to modify the relational algebra statement and choose an appropriate strategy for execution that reduces the storage, memory, computation, and networking cost for generating the result.
3. **Code Generation:** generate an executable code that will run in an interpreter or directly in the hardware against various database files to retrieve the result.
4. **Query Execution:** running the code to retrieve and return the result.



Query Compilation

The query is compiled into a relational algebra expression/statement as the first step of query processing as a relational algebra expression is a formal mathematical expression that is constructed in a recursive way and there are formal transformation rules that can be applied in this form to modify the query for performance improvement without changing the meaning of the query.

Furthermore, the executable code that is generated after query optimization uses building block parts for selection, join, projection, aggregate functions, etc. from relational algebra. The executable code is basically a way to glue existing implementations of these building block parts into a larger whole that answer the user query.

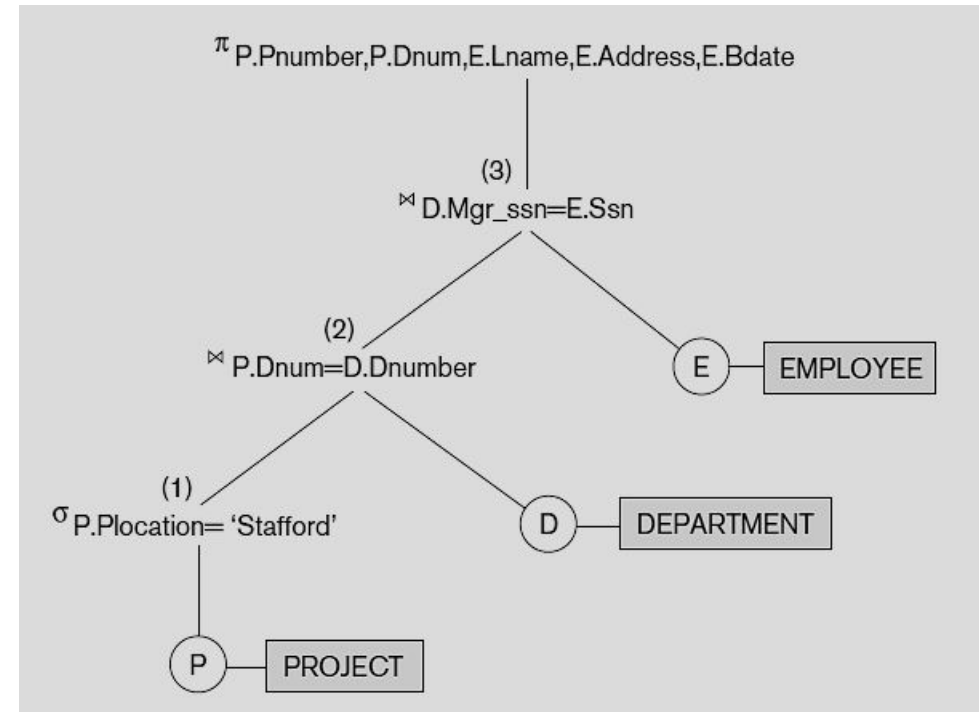


Query Tree Handling

The query processor in DBMS typically uses the tree representation of the relational algebra expression. The reason for using the query tree as opposed to the expression directly is that the tree format shows the order of operations and how intermediate results from earlier operations feed information to later operations. Hence, when combining building block codes for the basic operations, how that should be done to form an executable for the whole query is easy to decide from the tree representation. (Note that in the query tree, the leaves are always the original relations.)

Another advantage of the tree format is that most query optimization transformations are expressed as modifying/shuffling branches of the tree.

$\pi_{Pnumber, Dnum, Lname, Address, Bdate}(((\sigma_{Plocation='Stafford'}(PROJECT)) \bowtie_{Dnum=Dnumber}(DEPARTMENT)) \bowtie_{Mgr_ssn=Ssn}(EMPLOYEE))$



Key Algorithmic Components

The building block parts of the code that query processor stitches in are alternative algorithms for some key operations.

- Algorithms for external sorting.
- Algorithms for selection.
- Algorithms for Join.
- Algorithms for Projection.
- Algorithms for Set Operations.
- Algorithms for Aggregate Operations.

There are alternative algorithms for some basic operations because whether or not some algorithm can be applied for a particular query depends of a lot of things such as:

- Existence and nature of indexes on query related attributes.
- Volume and distribution of records in underlying relations.
- Relative fitness of an algorithm in the overall combining approach using query optimization. In other word, what combination will give be best performance for the overall query.

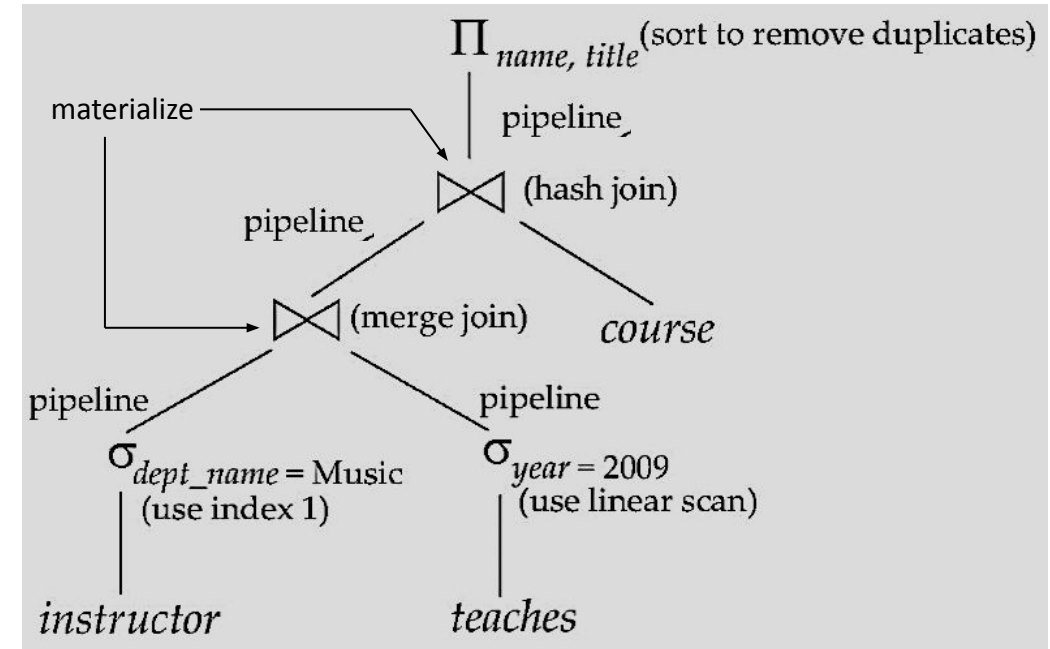
The Concept of Pipelining

One very important concepts of generating optimum executable code for a query is called query pipelining. The idea is that the code for building block operations will serves at stream/iterators that will return one tuple/record at a time until no more tuple exists. What will happen here is that the returned tuple will be processed by the next operation that is the parent of the current operation. It will further process the tuple and pass it higher up or may drop it and ask for the next tuple from its child depending on condition satisfactions.

Working in this manner allows the **tuples to go up one at a time from leaves to root** and being **modified as they move**. This drastically reduces the amount of buffering requirements in the intermediate blocks of the query tree, improving performance of single query and allow more concurrent query processing.

The limitation is that whenever there is a join, all tuples at least one branch of the join must be buffered to be matched with the tuples from the other branch. This is called **Materialization**.

When the data volume is too high, materialization may require intermediate files being stored in disks!

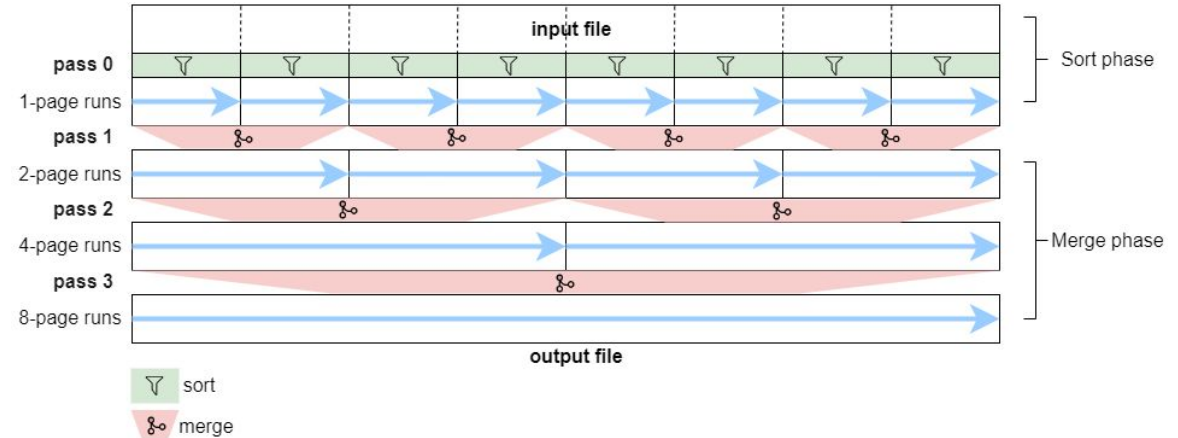


Which branch of a join is to be materialized, or whether both branches should be materialized itself is an interesting optimization question.

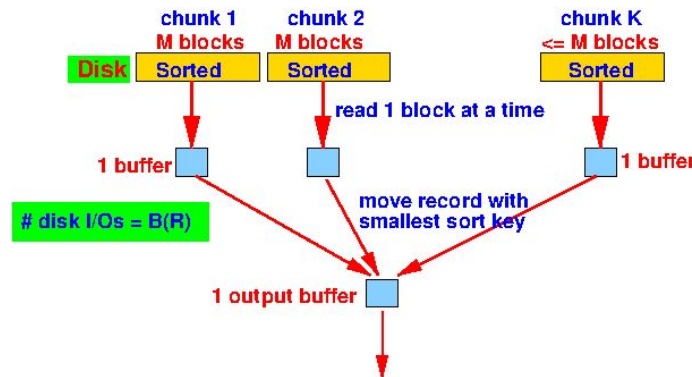
Techniques for External Sorting

External sorting applies when records/tuples are retrieved from the original file/files holding the records of a relation. Unless the searched attribute(s) is(are) the primary key, the records are not ordered by the attribute(s) of interest. Typically, a DBMS sorts records because sorting frequently made some subsequent operations such as join easier.

Regarding sorting, merge sort is ubiquitous in DBMS. Hardly any other sorting algorithm is being used. Regarding implementation of the merge sort, by the way, size of partitions to be merged are chosen based on block-size of the file-system or page-size of RAM, which is basically the unit of data being loaded from disk to RAM on a single disk read operation.



The implementation of merge sort is not the recursive implementation that goes down breaking things to pair of records. Similarly, not always only two sorted partitions are merged. Some cases, several sorted partitions are merged simultaneously.



General Strategies for Selections

There are several ways to implement a selection based on a single attribute. However, they broadly fall into two categories: table scan or index-based search.

Table scan means, scanning the database files for a relation and filter the tuples with matching values for the attribute of interest. This can be done two ways:

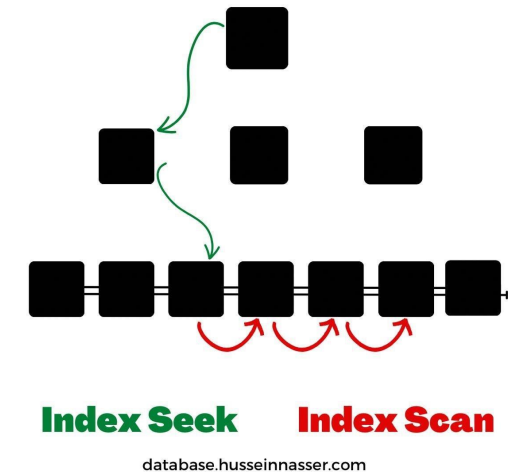
- Brute-forced: this is a linear scan of records from beginning to end of files.
- Binary search scan: scanning the files using binary search. This is only possible if the records are sorted in the files by the searched attribute.

Indexed-based search is possible when there is some indexing structure for the relation for the attribute being searched. Depending on the nature of the index, indexed-based search implementation differs.

- Primary Index
- Clustering Index
- Secondary B-tree Index
- Bitmap Index



Index-based search are generally better as they require reading significantly less data from files.



In case of tree-based indexes, when a value range ($<$, $<=$, $>$, $>=$) for the attribute is searched then the fact index itself is a sorted structure is cleverly utilized. Index seek leads to the first elements that satisfy the selection criteria. Then DBMS does index scan that keeps on reading subsequent leaves as long as the condition does not become false.

Conjunctive and Disjunctive Selections

When the selection is a disjunction (or) of several conditions then the typical way to implement is to use the conditions as a pipeline. Where each record is checked individually against each conditions and if any condition is found true the record is retained for passing up in the query tree. Otherwise, it is discarded.

When the selection is a conjunction (and) of several conditions, then multiple alternative exists.

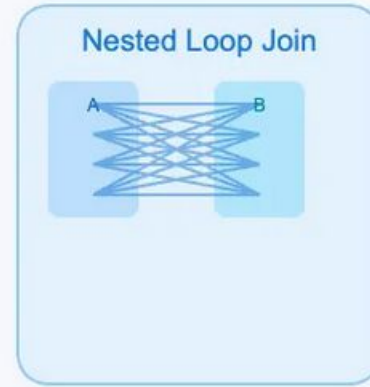
- Single Index Search: if there is an index on one of the attribute of selection conditions, then use that index to go through the matching records and then check other conditions on that records to decide whether to keep it or to discard it.
- Composite Index: if there is a composite index that uses all the attributes of the selection then do the selection on the index directly and just read the indexed records from files.
- Intersecting record pointers: do separate selection searches and maintain record pointers to file locations, then take the intersection of all pointer sets, finally return the records for the intersection.

However, conjunctive selection is rare. DBMS typically implements conjunctive selections as a chain of selections each operating on a single attribute. The reason is that doing so provide more opportunities for query optimization.

Strategies for Joins

Joins are often the most critical components related to query performance as when you combine records/tuples from multiple relations then you can have a huge blow up of data that DBMS need to hold and explore to implement the join. Furthermore, since the tuples for join come from two branches in the query tree, there is a need for materialization and pipelining must be broken in places of joins.

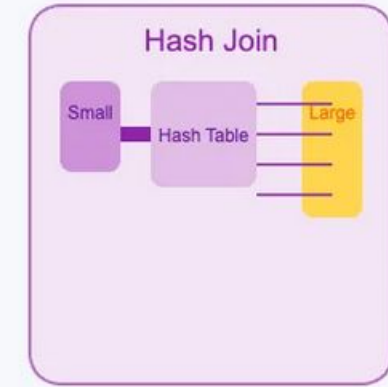
Typically, cartesian products are rare and almost always are converted to natural joins (inner, left, right, outer). Several alternative implementations are possible for these joins.



For each tuple on one branch, investigate all tuples in the other branch for matching. This is just like a doubly nested loop.



Sort the tuples of two branches separately then merge the sorted contents based on matching attribute. This process involved materialization on both sides.



Put the tuples of one branch in hash buckets. As tuples from the other branch arrives in pipeline fashion, check matching bucket. If a matching bucket is found then look for matching tuples in the bucket to form and pass up the combined tuples.

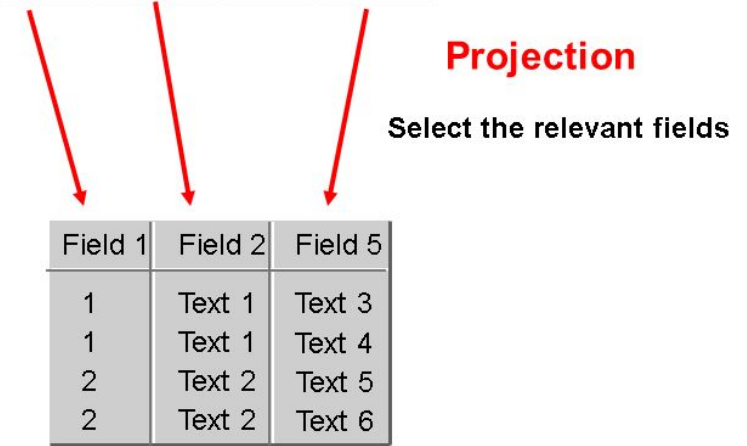
Index Use in Join Handling

- Typically, when the DBMS considers a particular implementation of join, the existence of indexing on relevant attributes typically does not matter. Indexing is most often used for implementing selection in the join branches before a join algorithm is applied.
- However, there is a special case when the attributes being used for joining the two relations have indexing in both relation.
- Then instead of retrieving tuples using select then applying one of the three join implementation algorithm, an index-based join can be implemented.
- In this case, the two indexing structures can be searched and then the intersection of indexing node values are computed. Then the combined tuples for join are produced from the indexing pointers to data files.

General Strategy for Projection

- Unlike other basic operations, there is only one strategy for implementing a projection.
- Iterative over the tuples returned from earlier step of the query tree and filter out the attributes that are not part of the projections.
- This is always done in an iterative fashion. Which means, that for each tuple being retrieved, projection is applied immediately. The algorithm does not collect a bunch of tuples from earlier step in any buffer then apply the filtering.
- Doing so would unnecessarily increase buffering overhead.

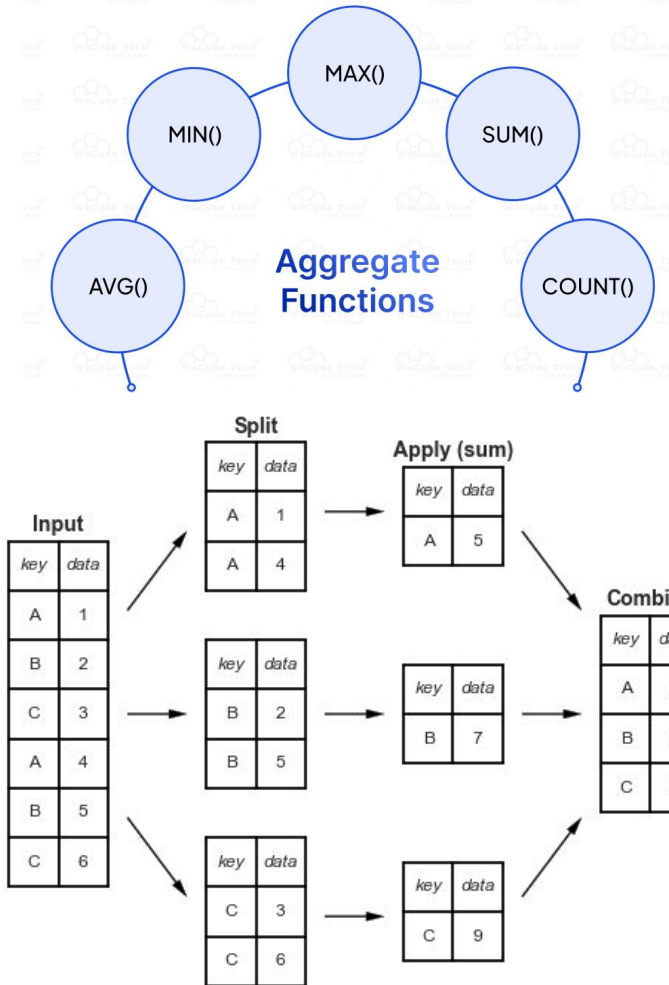
Field 1	Field 2	Field 4	Field 5
1	Text 1	A	Text 3
1	Text 1	B	Text 4
2	Text 2	A	Text 5
2	Text 2	B	Text 6



Strategy for Aggregate Functions

Generally, a single strategy is being used for implementing all aggregate operations. This is called **split-apply-combine** pipeline. The idea is the tuples from earlier stage/relation-file are split into bucket on the grouping attribute for different values. Then the aggregate function is applied in each group separately. Finally, the final values (one for each group) is merged.

Typically, the splitting is done in pipeline fashion. That is DBMS does not accumulate the tuples in separate buckets but, rather, when a new tuple arrives in a grouping bucket a running group function is updated.



Performance Concerns

When the data volume is low, even the most complicated queries return result quickly enough to the extent that which algorithm is being used for what basic operation implementation and how they are combined to generate the overall executable code for the input SQL query does not matter much. However, most real-world databases have a lot of data. Then the following cost concerns become relevant regarding query processing.

- **Storage Access:** the number of disk reads for record retrievals for index and data files and the number of read/write to disk for intermediate results holding.
- **Buffer consumptions:** the maximum memory consumption during the query execution due to buffering of intermediate results.
- **Computation time:** the time taken to run the whole query.
- **Communication Overhead:** the latency and bandwidth-usage for internal communication among DBMS parts during query execution.

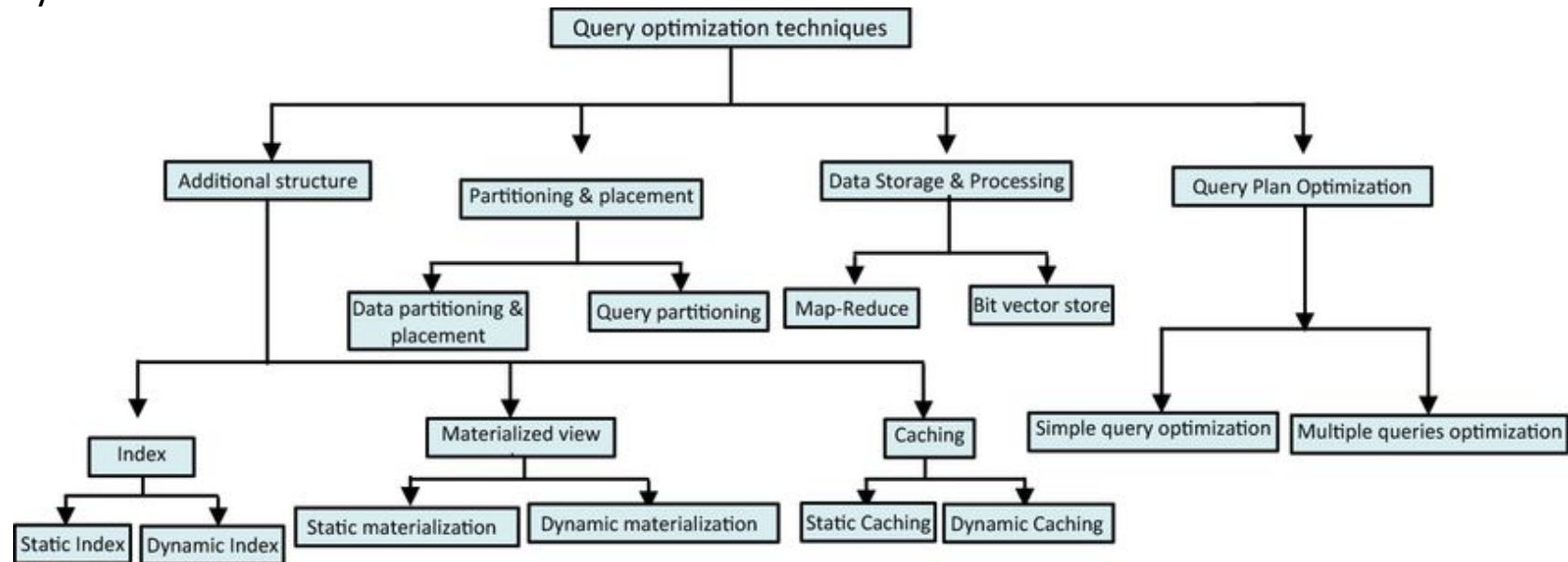
Since real-world databases need to handle hundred to thousands of queries concurrently, high cost of query processing is a great concern.

The Case for Query Optimization

It is possible that the difference in cost of an efficient query and an inefficient one that return the same result is several order of magnitude.

Therefore, query optimization is very important.

Optimizing important queries' performance is a join effort by the database administrators, application programmers, and the database management system.



DBMS vendors apply sophisticated algebraic heuristic rule and cost-based optimization in the query processing pipeline to optimize the query as much as possible.